HBase Lab

Beginner & Intermediate

Instructor-Led Practice + Student Exercises

Item	Details
Dataset	student table
Column Families	personal, academic
Levels Covered	Beginner → Intermediate

Dataset: Student Table

Column Families

Column Family	Columns
personal	name, city, age
academic	faculty, grade, year

Data

Row Key	Name	City	Age	Faculty	Grade	Year
1	ali	cairo	21	engineering	A	3
2	sara	alex	22	medicine	B	4
3	omar	giza	20	engineering	A	2
4	nada	cairo	23	arts	C	4
5	youssef	tanta	21	engineering	B	3

PART 1 — BEGINNER

1.1 Create the Table

Syntax:

```
create '<table>', '<column_family1>', '<column_family2>'
```

Instructor Demo:

```
create 'student', 'personal', 'academic'
```

Verify the table was created:

```
list
```

View table schema:

```
describe 'student'
```

1.2 Insert Data — put

Syntax:

```
put '<table>', '<row_key>', '<family:column>', '<value>'
```

Instructor Demo — Insert personal data:

```
put 'student', 1, 'personal:name', 'ali'
put 'student', 2, 'personal:name', 'sara'
put 'student', 3, 'personal:name', 'omar'
put 'student', 4, 'personal:name', 'nada'
put 'student', 5, 'personal:name', 'youssef'

put 'student', 1, 'personal:city', 'cairo'
```

```
put 'student', 2, 'personal:city', 'alex'
put 'student', 3, 'personal:city', 'giza'
put 'student', 4, 'personal:city', 'cairo'
put 'student', 5, 'personal:city', 'tanta'

put 'student', 1, 'personal:age', '21'
put 'student', 2, 'personal:age', '22'
put 'student', 3, 'personal:age', '20'
put 'student', 4, 'personal:age', '23'
put 'student', 5, 'personal:age', '21'
```

Instructor Demo — Insert academic data:

```
put 'student', 1, 'academic:faculty', 'engineering'
put 'student', 2, 'academic:faculty', 'medicine'
put 'student', 3, 'academic:faculty', 'engineering'
put 'student', 4, 'academic:faculty', 'arts'
put 'student', 5, 'academic:faculty', 'engineering'

put 'student', 1, 'academic:grade', 'A'
put 'student', 2, 'academic:grade', 'B'
put 'student', 3, 'academic:grade', 'A'
put 'student', 4, 'academic:grade', 'C'
put 'student', 5, 'academic:grade', 'B'

put 'student', 1, 'academic:year', '3'
put 'student', 2, 'academic:year', '4'
put 'student', 3, 'academic:year', '2'
put 'student', 4, 'academic:year', '4'
put 'student', 5, 'academic:year', '3'
```

1.3 Scan Data — scan

Syntax:

```
scan '<table>'
scan '<table>', { COLUMNS => '<family:col>', LIMIT => N, STARTROW => '<key>' }
```

Instructor Demo — Full scan:

```
scan 'student'
```

Instructor Demo — Scan with options:

```
scan 'student', {COLUMNS => 'personal:name', LIMIT => 3, STARTROW => '2'}
```

1.4 Get Data — get

Syntax:

```
get '<table>', '<row_key>'
get '<table>', '<row_key>', {COLUMN => '<family:col>'}
```

Instructor Demo — Get all columns of row 1:

```
get 'student', '1'
```

Instructor Demo — Get a specific column:

```
get 'student', '1', {COLUMN => 'academic:grade'}
```

1.5 Scan with Filter

Instructor Demo — Find students with grade A:

```
scan 'student', {
  COLUMNS => 'academic:grade',
  FILTER  => "ValueFilter(=,'binary:A')"
}
```

Student Exercises — Beginner

Use the student table you just built to answer the following. Write your commands in each answer box.

Q1. How many rows are in the student table?

Hint: There is a command that counts rows directly.

Your answer:

```
count 'student'
```

Q2. Get only the personal:city column for student with row key 3.

Your answer:

```
get 'student', '3', {COLUMN => 'personal:city'}
```

Q3. Scan the table and return only academic:faculty, starting from row 2, with a limit of 3 rows.

Your answer:

```
scan 'student', {COLUMNS => 'academic:faculty', STARTROW => '2', LIMIT => 3}
```

```
hbase(main):056:0> scan 'student', {COLUMNS => 'personal:name', LIMIT => 3, STARTROW => '2'}
ROW                                   COLUMN+CELL
 2                                    column=personal:name, timestamp=2026-05-23T13:49:42.356, value=sara
 3                                    column=personal:name, timestamp=2026-05-23T13:49:42.395, value=omar
 4                                    column=personal:name, timestamp=2026-05-23T13:49:42.441, value=nada
3 row(s)
```

Q4. Student 4 (nada) has moved from cairo to mansoura. Update her city.
Hint: put overwrites an existing value.

Your answer:

```
put 'student', '4', 'personal:city', 'mansoura'
```

Q5. Delete only the personal:age cell for student 3.
Hint: Look up the delete command.

Your answer:

```
delete 'student', '3', 'personal:age'
```

Q6. Confirm the deletion worked by getting all data for row 3.

Your answer:

```
get 'student', '3'
```

Q7. Delete the entire row for student 5 (youssef).
Hint: Look up deleteall.

Your answer:

```
deleteall 'student', '5'
```

Q8. Verify student 5 no longer exists with a full scan.

Your answer:

```
scan 'student'
```

```
hbase(main):064:0> scan 'student'
ROW                                     COLUMN+CELL
 1                                      column=academic:faculty, timestamp=2026-05-23T13:50:24.270, value=engineering
 1                                      column=academic:grade, timestamp=2026-05-23T13:50:24.416, value=A
 1                                      column=academic:year, timestamp=2026-05-23T13:50:24.545, value=3
 1                                      column=personal:age, timestamp=2026-05-23T13:49:42.695, value=21
 1                                      column=personal:city, timestamp=2026-05-23T13:49:42.516, value=cairo
 1                                      column=personal:name, timestamp=2026-05-23T13:49:42.298, value=ali
 2                                      column=academic:faculty, timestamp=2026-05-23T13:50:24.298, value=medicine
 2                                      column=academic:grade, timestamp=2026-05-23T13:50:24.440, value=B
 2                                      column=academic:year, timestamp=2026-05-23T13:50:24.570, value=4
 2                                      column=personal:age, timestamp=2026-05-23T13:49:42.741, value=22
 2                                      column=personal:city, timestamp=2026-05-23T13:49:42.549, value=alex
 2                                      column=personal:name, timestamp=2026-05-23T13:49:42.356, value=sara
 3                                      column=academic:faculty, timestamp=2026-05-23T13:50:24.331, value=engineering
 3                                      column=academic:grade, timestamp=2026-05-23T13:50:24.465, value=A
 3                                      column=academic:year, timestamp=2026-05-23T13:50:24.601, value=2
 3                                      column=personal:age, timestamp=2026-05-23T13:49:42.789, value=20
 3                                      column=personal:city, timestamp=2026-05-23T13:49:42.583, value=giza
 3                                      column=personal:name, timestamp=2026-05-23T13:49:42.395, value=omar
 4                                      column=academic:faculty, timestamp=2026-05-23T13:50:24.352, value=arts
 4                                      column=academic:grade, timestamp=2026-05-23T13:50:24.488, value=C
 4                                      column=academic:year, timestamp=2026-05-23T13:50:24.623, value=4
 4                                      column=personal:age, timestamp=2026-05-23T13:49:42.818, value=23
 4                                      column=personal:city, timestamp=2026-05-23T14:06:49.556, value=mansoura
 4                                      column=personal:name, timestamp=2026-05-23T13:49:42.441, value=nada
 5                                      column=academic:faculty, timestamp=2026-05-23T13:50:24.378, value=engineering
 5                                      column=academic:grade, timestamp=2026-05-23T13:50:24.514, value=B
 5                                      column=academic:year, timestamp=2026-05-23T13:50:24.644, value=3
 5                                      column=personal:age, timestamp=2026-05-23T13:49:42.843, value=21
 5                                      column=personal:city, timestamp=2026-05-23T13:49:42.654, value=tanta
 5                                      column=personal:name, timestamp=2026-05-23T13:49:42.472, value=youssef
5 row(s)
```

PART 2 — INTERMEDIATE

2.1 Versioning

By default, HBase keeps 1 version of each cell. You can increase this to store the full history of changes.

Alter the table to keep 3 versions of the academic column family:

```
alter 'student', NAME => 'academic', VERSIONS => 3
```

Instructor Demo — Update a grade multiple times:

```
put 'student', 1, 'academic:grade', 'B'
put 'student', 1, 'academic:grade', 'A+'
```

Instructor Demo — Get the latest version (default):

```
get 'student', '1', {COLUMN => 'academic:grade'}
```

Instructor Demo — Get all 3 versions:

```
get 'student', '1', {COLUMN => 'academic:grade', VERSIONS => 3}
```

> *Each value is shown with its own timestamp — you can trace the full history of changes.*

2.2 Filters

HBase supports powerful server-side filters to query data efficiently.

PrefixFilter — Rows starting with a value

```
scan 'student', {FILTER => "PrefixFilter('1')"}
```

RowFilter — Match an exact row key

```
scan 'student', {FILTER => "RowFilter(=,'binary:2')"}
```

SingleColumnValueFilter — Filter by column value

Returns the full row where a specific column matches a value.

Instructor Demo — Find all engineering students:

```
scan 'student', {
  FILTER => "SingleColumnValueFilter('academic','faculty',=,'binary:engineering')"
}
```

Instructor Demo — Find students with grade B:

```
scan 'student', {
  FILTER => "SingleColumnValueFilter('academic','grade',=,'binary:B')"
}
```

ColumnPrefixFilter — Columns starting with a prefix

```
scan 'student', {FILTER => "ColumnPrefixFilter('gr')"}
```

2.3 Alter Table — Add a Column Family

Instructor Demo — Add a new column family contact:

```
alter 'student', NAME => 'contact'
```

Add data to the new column family:

```
put 'student', 1, 'contact:email', 'ali@uni.edu'
put 'student', 2, 'contact:email', 'sara@uni.edu'
```

Verify the schema changed:

```
describe 'student'
```

2.4 Table Lifecycle

```
disable 'student'     # required before dropping
is_disabled 'student' # check status
enable 'student'      # re-enable

disable 'student'     # drop requires disabled
drop 'student'
list                  # confirm it's gone
```

⚠ *You cannot drop a table that is still enabled.*

Student Exercises — Intermediate

Think carefully before running each command. Write your answers in each box.

Q1. Enable versioning on the personal column family — keep up to 5 versions.

Your answer:

```
alter 'student', {NAME => 'personal', VERSIONS => 5}
```

```
hbase(main):076:0* get 'student', '2', {COLUMN => 'personal:city', VERSIONS => 3
}
COLUMN                CELL
 personal:city        timestamp=2026-05-24T11:08:30.679, value=alex
 personal:city        timestamp=2026-05-24T11:08:30.663, value=giza
 personal:city        timestamp=2026-05-24T11:08:30.648, value=cairo
1 row(s)
Took 0.0119 seconds
```

Q2. Update personal:city for student 2 (sara) three times: first to cairo, then giza, then back to alex. Then retrieve all versions of that cell.

Hint: Use put three times, then get with VERSIONS => 3.

Your answer:

```
put 'student', '2', 'personal:city', 'cairo'
put 'student', '2', 'personal:city', 'giza'
put 'student', '2', 'personal:city', 'alex'


get 'student', '2', {COLUMN => 'personal:city', VERSIONS => 3}
```

Q3. Use SingleColumnValueFilter to find all students in year 4.

Your answer:

```
scan 'student', {FILTER => "SingleColumnValueFilter('academic', 'year', =, 'binary:4')"}
```

Q4. Use SingleColumnValueFilter to find all students who live in cairo.

Your answer:

```
scan 'student', {FILTER => "SingleColumnValueFilter('personal', 'city', =, 'binary:cairo')"}
```

Q5. Add contact:phone for students 1, 3, and 4 with any phone numbers you choose.

Your answer:

```
put 'student', '1', 'contact:phone', '01012345678'
put 'student', '3', 'contact:phone', '01198765432'
put 'student', '4', 'contact:phone', '0125551234'
```

Q6. Scan the table and return only the contact column family for all students.

Your answer:

```
scan 'student', {COLUMNS => 'contact'}
```

Q7. Add a new column family called enrollment to the student table, then verify it appears in the schema.

Your answer:

```
alter 'student', {NAME => 'enrollment'}
describe 'student'
```

Q8. CHALLENGE Write a scan that returns only students who are in the engineering faculty AND in year 3.

Hint: Look up FilterList and how to combine filters with MUST_PASS_ALL.

Your answer:

```
 scan 'student} ,'
  FILTER => "SingleColumnValueFilter('academic','faculty',=,'binary:engineering') AND
SingleColumnValueFilter('academic', 'year', =,'binary:3')"}
```